

ZJOI2026 模拟赛

第二试

hcywoi & Milkcat

时间：2026 年 3 月 5 日 07:30 ~ 12:30

题目名称	或加或减	战略相持	树
题目类型	传统型	传统型	传统型
目录	pom	stalemate	tree
可执行文件名	pom	stalemate	tree
输入文件名	pom.in	stalemate.in	tree.in
输出文件名	pom.out	stalemate.out	tree.out
每个测试点时限	2.0 秒	3.0 秒	3.0 秒
内存限制	1024 MiB	1024 MiB	1024 MiB
数据点数目	6	25	5
数据点是否等分	否	是	否

提交源程序文件名

对于 C++ 语言	pom.cpp	stalemate.cpp	tree.cpp
-----------	---------	---------------	----------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 文件名（包括程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 main() 的返回值类型必须是 int，值必须为 0。
3. 对于因未遵守以上规则对成绩造成的影响，相关申诉不予受理。
4. 若无特殊说明，输入文件中同一行内的多个整数、浮点数、字符串等均使用一个空格进行分隔。
5. 若无特殊说明，结果比较方式为忽略行末空格、文末回车后的全文比较。
6. 程序可使用的栈空间大小与该题内存空间限制一致。
7. 在终端下可使用命令 `ulimit -s unlimited` 将栈空间限制放大，但你使用的栈空间大小不应超过题目限制。
8. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

或加或减 (pom)

【题目描述】

你有两个 n 位二进制数 A, B (可能有前导 0) 和一个非负整数序列 $c_0, c_1, \dots, c_{n-1}$ 。你可以进行若干次如下操作：

- 选择满足 $|A - C| = 2^k$ 且 $k < n$ 的非负整数 C, k ，然后将 A 变为 $A \text{ bitor } C$ ，代价为 c_k 。

其中 bitor 表示二进制按位或。

你需要通过若干次操作使得 A 变为 B ，求出最小的代价和。若无解则输出 -1 。

【输入格式】

从文件 `pom.in` 中读入数据。

本题输入包含多组数据。

第一行，一个整数 T ，表示数据组数。对于每组数据：

- 第一行，一个正整数 n 。
- 第二行，一个长度为 n 的 01 串，表示 A 二进制下从高到低的数位。
- 第三行，一个长度为 n 的 01 串，表示 B 二进制下从高到低的数位。
- 第四行， n 个非负整数 $c_{n-1}, c_{n-2}, \dots, c_0$ ，**注意此处是倒序的**。

【输出格式】

输出到文件 `pom.out` 中。

对于每组测试数据，输出一行，一个整数，表示答案。

【样例 1 输入】

```
1 8
2 3
3 000
4 000
5 1 2 3
6 3
7 000
8 101
9 1 2 3
10 3
```

```
11 000
12 111
13 1 2 3
14 3
15 000
16 111
17 3 2 1
18 6
19 000100
20 111101
21 1 2 2 1 1 1
22 6
23 001001
24 111111
25 8 4 7 3 6 2
26 10
27 0000000000
28 1111111111
29 1 1 4 5 1 4 1 9 1 9
30 10
31 1111111111
32 0000000000
33 1 1 4 5 1 4 1 9 1 9
```

【样例 1 输出】

```
1 0
2 4
3 4
4 3
5 4
6 8
7 10
8 -1
```

【样例 1 解释】

对于第一组数据，不需要任何操作，故答案为 0。

对于第二组数据，两次操作分别为：

- 选择 $C = 1$, $|A - C| = 2^0$, 代价为 $c_0 = 3$, 操作后 $A = 1$;
- 选择 $C = 5$, $|A - C| = 2^2$, 代价为 $c_2 = 1$, 操作后 $A = 5$ 。

可以证明不存在代价更小的操作方法，故答案为 4。

对于第三组数据，两次操作分别为：

- 选择 $C = 4$, $|A - C| = 2^2$, 代价为 $c_2 = 1$, 操作后 $A = 4$;
- 选择 $C = 3$, $|A - C| = 2^0$, 代价为 $c_0 = 3$, 操作后 $A = 7$ 。

可以证明不存在代价更小的操作方法，故答案为 4。

【样例 2】

见选手目录下的 *pom/pom2.in* 与 *pom/pom2.ans*。

该样例满足第 3 个子任务的性质。

【样例 3】

见选手目录下的 *pom/pom3.in* 与 *pom/pom3.ans*。

该样例满足第 4 个子任务的性质。

【样例 4】

见选手目录下的 *pom/pom4.in* 与 *pom/pom4.ans*。

该样例满足第 5 个子任务的性质。

【样例 5】

见选手目录下的 *pom/pom5.in* 与 *pom/pom5.ans*。

该样例满足第 7 个子任务的性质。

【样例 6】

见选手目录下的 *pom/pom6.in* 与 *pom/pom6.ans*。

该样例满足第 8 个子任务的性质。

【数据范围】

本题开启捆绑测试。

对于所有数据， $1 \leq n, T \leq 10^5$ ， $\sum n \leq 10^6$ ， $0 \leq c_i \leq 10^9$ ，输入 A, B 时仅含字符 01。

各子任务的附加限制如下表所示：

测试点编号	$n \leq$	$T \leq$	特殊性质	分值
1	3	100	无	5
2	13			15
3	10^5	10^5	BC	5
4			BD	15
5	10^3	20	AB	15
6	10^5	10^5		20
7	10^3	20	无	10
8	10^5	10^5		15

特殊性质：

- 特殊性质 A： $A = 0$ 。
- 特殊性质 B： $B = 2^n - 1$ 。
- 特殊性质 C：对于所有 $i \in [0, n)$ ， $c_i = 1$ 。
- 特殊性质 D：对于所有 $i \in [0, n)$ ， $c_i \leq 2$ 。

战略相持 (stalemate)

【题目描述】

A 国由 n 座城市组成，有 m 条连接两个城市的无向道路。

敌对国家 B 将要侵略 A 国，并占领一些道路。若一个城市相邻的所有道路被占领，则这个城市也被占领。

一种占领方案中，设未被占领的道路集合是 E ，与 E 中至少一条边相邻的城市集合为 S 。如果以下条件满足，两国的战争就会进入“战略相持阶段”：

- $|S| = |E|$;
- 对于任意 $x, y \in S$ ，存在一条只经过 E 中道路的连接 x, y 的路径。

在这种情况下，称一座城市是安全的，当且仅当它与 E 中至少两条道路相邻。给定数组 h ，若安全城市数量为 k ，则这种占领方案的权值为 h_k 。

现在，A 国元首小 A 有 q 次询问，每次他会给你数组 h ，你需要求出所有可能的 2^m 种占领方案中，使战争进入“战略相持阶段”的方案的权值和。答案对 998244353 取模。

【输入格式】

从文件 *stalemate.in* 中读入数据。

第一行三个正整数 n, m, q ，含义如题意所述。

接下来的 m 行，第 i 行有两个整数 x, y ，表示城市 x 与城市 y 之间有一条无向道路。

接下来 q 行，每行 n 个非负整数表示询问的数组 h 。

【输出格式】

输出到文件 *stalemate.out* 中。

输出 q 行，每行一个非负整数表示答案对 998244353 取模后的值。

【样例 1 输入】

```
1 4 4 4
2 1 2
3 1 3
4 2 3
5 1 4
6 1 0 0 0
7 0 1 0 0
8 0 0 1 0
```

```
9 0 0 0 1
```

【样例 1 输出】

```
1 0
2 0
3 2
4 0
```

【样例 1 解释】

两种可以使战争进入“战略相持阶段”的占领方案分别为：不占领任何道路；占领道路 (1,4)。两种方案中安全城市均为 1,2,3。

【样例 1 输入】

```
1 5 5 5
2 1 2
3 1 3
4 2 3
5 1 4
6 4 5
7 1 0 0 0 0
8 0 1 0 0 0
9 0 0 1 0 0
10 0 0 0 1 0
11 0 0 0 0 1
```

【样例 1 输出】

```
1 0
2 0
3 2
4 1
5 0
```

【样例 2 解释】

三种可以使战争进入“战略相持阶段”的占领方案分别为：不占领任何道路；占领道路 (4, 5)；占领道路 (1, 4) 与 (4, 5)。

第一种方案数中安全城市为 1, 2, 3, 4，后两种方案中安全城市为 1, 2, 3。

【样例 3】

见选手目录下的 *stalemate/stalemate3.in* 与 *stalemate/stalemate3.ans*。
该样例满足第 4 个测试点的性质。

【样例 4】

见选手目录下的 *stalemate/stalemate4.in* 与 *stalemate/stalemate4.ans*。
该样例满足第 13, 14 个测试点的性质。

【样例 5】

见选手目录下的 *stalemate/stalemate5.in* 与 *stalemate/stalemate5.ans*。
该样例满足第 15, 16 个测试点的性质。

【样例 6】

见选手目录下的 *stalemate/stalemate6.in* 与 *stalemate/stalemate6.ans*。
该样例满足第 18 个测试点的性质。

【样例 7】

见选手目录下的 *stalemate/stalemate7.in* 与 *stalemate/stalemate7.ans*。
该样例满足第 24, 25 个测试点的性质。

【数据范围】

对于所有数据， $1 \leq n \leq 19$ ， $1 \leq m \leq \frac{n(n-1)}{2}$ ， $1 \leq q \leq 10$ ， $1 \leq x, y \leq n$ ， $x \neq y$ ，不存在两条相同的道路。

各测试包的附加限制如下表所示：

测试点编号	$n \leq$	$q \leq$	特殊性质
1	5	10	无
2	11	1	A
3			B
4			无
5, 6	14	1	A
7			B
8			无
9, 10	15	1	A
11			B
12			无
13, 14	16	1	A
15, 16			B
17			无
18		10	
19	18	1	A
20			B
21			无
22		10	
23		1	
24, 25	19	10	

- 特殊性质：
- 特殊性质 A： $h_1 = h_2 = \cdots = h_n = 1$ ；
 - 特殊性质 B： 对于所有 $i \in [1, n]$ ， $h_i \times 2^i \equiv 1 \pmod{998244353}$ 。

树 (tree)

【题目描述】

有一棵 n 个点、根为 1 的树。第 i 个点上有 c_i 个物品，权值均为 v_i 。

需要先选择 t 个不同的点，对于选的每个点，在它的每个祖先（包括自己）上各取一个物品。本题保证 $t < c_i$ ，因此不会存在没有物品可取的情况。

接下来可以取至多 k 个物品，要求所有在这一步被取了物品的点形成一个含根的连通块。

最大化整个过程中取的物品权值之和。

【输入格式】

从文件 `tree.in` 中读入数据。

第一行三个整数 n, k, t 。

接下来 n 行，每行两个正整数分别表示 c_i 和 v_i 。

最后一行 $n - 1$ 个正整数，第 i 个正整数表示点 $(i + 1)$ 在树上的父亲 fa_{i+1} 。

【输出格式】

输出到文件 `tree.out` 中。

一行，输出一个非负整数，表示最大的权值和。

【样例 1 输入】

```
1 4 4 1
2 2 1
3 2 1
4 2 5
5 2 1
6 1 1 2
```

【样例 1 输出】

```
1 15
```

【样例 2】

见选手目录下的 `tree/tree2.in` 与 `tree/tree2.ans`。

【样例 3】

见选手目录下的 *tree/tree3.in* 与 *tree/tree3.ans*。

【样例 4】

见选手目录下的 *tree/tree4.in* 与 *tree/tree4.ans*。

【数据范围】

本题开启捆绑测试。

- 子任务 1 (10 分): $n, k, t \leq 10$ 。
- 子任务 2 (20 分): $t = 0$ 。
- 子任务 3 (15 分): $nk(t+1) \leq 10^6$ 。
- 子任务 4 (25 分): $nk(t+1) \leq 5 \times 10^6$ 。
- 子任务 5 (30 分): 无特殊限制。

对于所有数据满足, $1 \leq n, k \leq 10^4$, $0 \leq t \leq n$, $t < c_i \leq 10^9$, $1 \leq v_i \leq 10^9$, $\text{fa}_i < i$, $nk(t+1) \leq 10^7$ 。